
LAURUS

Provable Security Analysis

June 5, 2026

Contents

1	Introduction	2
1.1	The NGCC Program	2
1.2	The Laurus Hash Algorithm	2
1.3	Organization	3
2	Preliminaries	3
2.1	Notations	3
2.2	Hash Function and Security Definitions	4
2.3	Indifferentiability Framework	5
3	The Operation Mode Laurus^{δ}	6
3.1	Building Blocks	6
3.2	Main Construction	7
3.3	Generalized VFB Mode	8
4	Remaining Security Proofs	11
4.1	Collision Resistance for $\lambda = 1024$	11
4.2	(Second) Preimage Resistance for $\lambda = 512, 768, 1024$	13
5	Conclusions	13

1 Introduction

1.1 The NGCC Program

In October 2025, the Institute of Commercial Cryptography Standards (ICCS) launched the Next-generation Commercial Cryptographic Algorithms Program (NGCC), soliciting submissions for new cryptographic hash algorithms [Ins25]. The program aims to select and standardize hash functions that provide robust security against both classical and quantum adversaries, support message digest lengths of 512 and 1024 bits with an optional 768-bit variant, accommodate messages of up to $2^{64} - 1$ bits, and demonstrate efficient performance across a wide range of software and hardware platforms.

The NGCC submission requirements [Ins25] mandate that each submission be accompanied by a complete written specification covering algorithm description, design rationale, security statements and analyses both theoretical and practical, performance evaluation, and feature statements. In particular, for any given security parameter λ , the submission must provide rigorous proofs of resistance against *collision*, *preimage*, and *second preimage* attacks, with concrete security bounds expressed in terms of λ and the algorithm’s structural parameters. This document constitutes the *provable security statements and analysis* component of the Laurus submission package. In addition to the three required attack types, we establish an indistinguishability security bound for Laurus, providing a structural guarantee beyond the concrete security proofs.

1.2 The Laurus Hash Algorithm

Laurus is a permutation-based iterative hash function submitted to the NGCC program. Its operation mode, illustrated in Fig. 1, is a variant of the VFB mode [LLHL26], which is the first permutation-based hashing mode to achieve security bounds approaching the *optimal* $b/2$ bits, where b is the permutation width.

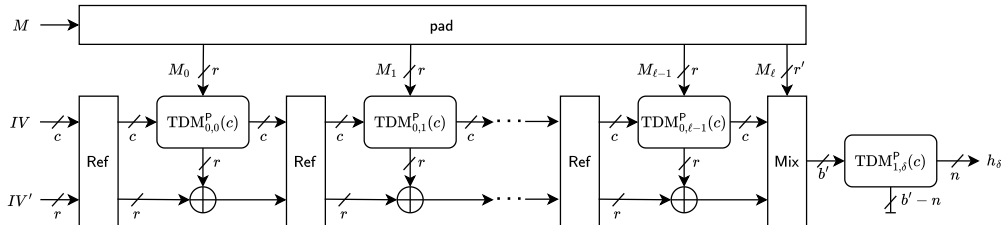


Figure 1. The operation mode of Laurus^δ.

Permutation-based hash functions have grown increasingly popular since the introduction of the sponge construction [BDPA07] and its standardization as SHA-3 (Keccak) [BDPA09]. The sponge partitions the internal state into a *rate* r and a *capacity* c , and outputs an n -bit digest. It achieves collision resistance of $\min\{n/2, c/2\}$ bits, preimage resistance of $\min\{n, \max\{c/2, n - r\}\}$ bits [LM22], and indistinguishability security bounded by $c/2$ bits [BDPA08]. Beyond hashing, sponge-based designs are now widely deployed in public-key encryption and authenticated encryption; the sponge-based Ascon [DEMS21] was selected as the NIST lightweight cryptography standard. Recent years have witnessed a surge of new permutation-based hash modes, including Sponge-DM and Sponge-EDM [SLZ⁺25] which boost preimage security to n bits, and Sponge-F [GHJ⁺25] which achieves similar gains under non-idealized assumptions. In particular, the double sponge [LM24], the VFB construction [LLHL26], and the alternating sponge [LSY⁺26] push the indistinguishability security beyond the classical birthday bound.

To achieve efficient output across the required parameter sets, Laurus increases the feed-forward length and replaces the finalization function of VFB with one that supports output lengths up to 1024 bits. To support hash and extendable-output function (XOF) modes at different security levels, we consider the core building block $\text{Laurus}^\delta[c, r, n]$, where c is the capacity, r is the rate, n is the length of output, and the parameter δ selects which member of a family of finalization functions is used for output generation. Each instantiation of Laurus is obtained by concatenating the outputs of several invocations of $\text{Laurus}^\delta[c, r, n]$ and then truncating the result to the desired length. The following informal theorem summarizes our main security results.

Theorem 1.1 (Informal). For any non-negative integer δ , the $\text{Laurus}^\delta[c, r, n]$ construction achieves collision resistance of $\min\{n/2, c/2\}$ bits, preimage resistance of $\min\{n, c\}$ bits, and second preimage resistance of $\min\{n, c\}$ bits. Moreover, for every $\lambda \in \{512, 768, 1024\}$, there exist parameters $c = c(\lambda)$, $r = r(\lambda)$, and $n = n(\lambda)$ such that $\text{Laurus}^\delta[c, r, n]$ is indistinguishable from an n -bit random oracle up to 512 bits. In particular, $\text{Laurus}^\delta[512, 1024, 1024]$ is indistinguishable from a 1024-bit random oracle up to 512 bits.

In the reference instantiation, Laurus employs a permutation of width $b = 1600$ bits as its underlying primitive and controls the iteration via a $\theta = 64$ -bit counter. Let $b' = b - \theta$ denote the effective width of the permutation. As a hash function, Laurus is instantiated as $\text{Laurus}^0[\lambda, b' - \lambda, \lambda]$ for $\lambda \in \{512, 768, 1024\}$. Consequently, the Laurus hash achieves *optimal* collision, preimage, and second-preimage resistance, while its indistinguishability bound *far exceeds* that of existing variants including the standard sponge construction. As an XOF, Laurus is obtained by concatenating the outputs of $\text{Laurus}^\delta[512, 1024, 1024]$ for $\delta \in \{1, 2, \dots, L\}$; at the 512-bit security level it likewise attains *optimal* security for all three resistance properties and *optimal indistinguishability*. We therefore believe that Laurus is a compelling candidate for the next-generation hash function standard.

1.3 Organization

The remainder of this document is organized as follows. [Sec. 2](#) introduces notation, cryptographic definitions, and the indistinguishability framework. [Sec. 3](#) provides the complete specification of Laurus and the HVFB working mode, including the building blocks, the main construction, and a detailed comparison with the VFB mode on which it is based. [Sec. 4](#) contains the remaining security proofs for the parameter regimes not covered by the VFB reduction: collision resistance for $\lambda = 1024$, (second) preimage resistance for $\lambda \in \{512, 768, 1024\}$. Finally, [Sec. 5](#) compares Laurus with existing permutation-based hash functions and concludes with a summary.

2 Preliminaries

2.1 Notations

Let $\mathbb{F}_2 = \{0, 1\}$ be the finite field of two elements. For a positive integer n , let $\mathbb{F}_2^n$ be the n -dimensional vector space over $\mathbb{F}_2$ equipped with the vector addition $+$, i.e., the bitwise XOR. For $x = (x_0, x_1, \dots, x_{n-1}) \in \mathbb{F}_2^n$, we denote its bit length by $|x| := n$. Let $[n] = \{0, 1, \dots, n-1\}$ and $(n)_2$ be the binary representation of n . For indices $a, b \in [n]$ with $a \leq b$, we define the sub-vector $x[a, b] := (x_{a-1}, x_a, \dots, x_{b-1}) \in \mathbb{F}_2^{b-a+1}$. For $x \in \mathbb{F}_2^n$ and $y \in \mathbb{F}_2^m$, we denote their concatenation by $x||y := (x, y) \in \mathbb{F}_2^{n+m}$.

For any set S , we denote its cardinality by $\#S$, write $s \xleftarrow{\$} S$ to indicate sampling s uniformly from S , and define

$$S^\infty := \bigcup_{n \geq 1} S^n,$$

where S^n is the Cartesian product of n copies of S . For sets X and Y , let $\text{Fun}(X, Y)$ be the set of all functions from X to Y . In particular, we write $\text{RO} \stackrel{\$}{\leftarrow} \text{Fun}(X, Y)$ to represent a random oracle sampled uniformly from $\text{Fun}(X, Y)$. If $X = Y = \mathbb{F}_2^b$ for some $b \in \mathbb{N}$, we denote by $\text{Perm}(b) \subset \text{Fun}(\mathbb{F}_2^b, \mathbb{F}_2^b)$ the set of all permutations over $\mathbb{F}_2^b$.

For real-valued functions $f(n)$ and $g(n)$, we write $f(n) = \mathcal{O}(g(n))$ if there exist constants $c, n_0 > 0$ such that for all $n \geq n_0$, $|f(n)| \leq c \cdot g(n)$. Similarly, $f(n) = o(g(n))$ if for any constant $c > 0$, there exists $n_0 > 0$ such that for all $n \geq n_0$, $|f(n)| \leq c \cdot g(n)$. A function $f(n)$ is negligible, denoted $f(n) = \text{negl}(n)$, if $f(n) = o(n^{-c})$ for every constant $c > 0$.

2.2 Hash Function and Security Definitions

Definition 2.1 (ϵ -Saturated Padding Function [LLHL26, Definition 1]). An injective function

$$\text{pad} : \mathbb{F}_2^\infty \rightarrow (\mathbb{F}_2^r)^\infty \times \mathbb{F}_2^{r'}, M \mapsto ((M_0, M_1, \dots, M_{\ell-1}), M_\ell)$$

is called an **ϵ -saturated padding** if the following conditions hold:

- (1) Let π_i be the projection that maps a padded message to its i -th r -bit block. The image of $\pi_i \circ \text{pad}$ contains at least $2^{r-\epsilon}$ distinct elements in $\mathbb{F}_2^r$ for any $0 \leq i \leq \ell - 1$.
- (2) There exists a polynomial-time algorithm Depad such that for any valid input in the range of pad , $\text{Depad}(\text{pad}(M)) = M$. For invalid inputs, it outputs $\perp$.
- (3) For every block count $\ell \geq 1$, there exists a final block $M_f = M_f(\ell) \in \mathbb{F}_2^{r'}$ such that, when $M_0, \dots, M_{\ell-1}$ are sampled uniformly and independently from $\mathbb{F}_2^r$,

$$\mathbb{P}\left[M_i \stackrel{\$}{\leftarrow} \mathbb{F}_2^r \ (0 \leq i \leq \ell - 1); \text{Depad}((M_0, \dots, M_{\ell-1}), M_f) \neq \perp\right] \geq 2^{-\epsilon(\ell+1)}.$$

We refer to $\epsilon \geq 0$ as the **deficiency**, $r \geq 1$ as the **rate**, and $r' \geq 0$ as the **tail length** of pad . The set of all such padding functions is denoted by $\text{Pad}(\epsilon, r, r')$.

Definition 2.2 (Iterated Hash Function [LLHL26, Definition 2]). An iterated hash function $H \in \text{Hash}(r, r', n, m, \text{IV})$ consists of a pair $(H.\text{pad}, H.\text{val})$, where $H.\text{pad} \in \text{Pad}(\epsilon, r, r')$ for some $\epsilon \leq r$, and $H.\text{val}$ is the composition of a compression function $F : \mathbb{F}_2^{m+r} \rightarrow \mathbb{F}_2^m$ and a chop function $G : \mathbb{F}_2^{m+r'} \rightarrow \mathbb{F}_2^n$. Let $((M_0, \dots, M_{\ell-1}), M_\ell)$ denote the image of a message M under pad . Formally,

$$H(M) = G(F(\dots F(F(\text{IV}, M_0), M_1), \dots, M_{\ell-1}), M_\ell).$$

In the following, we assume that the underlying primitive of the iterated hash function H is a single permutation P .

Definition 2.3 (Collision, Preimage, and Second Preimage Resistance). Let $H \in \text{Hash}(r, r', n, m, \text{IV})$ be an iterated hash function, and λ denote the security parameter. For any adversary $\mathcal{A}$, we define

$$\text{Adv}_{H, \mathcal{A}}^{\text{coll}}(\lambda) := \mathbb{P}_P \left[(M, M') \leftarrow \mathcal{A}^{H^P, P^\pm}(1^\lambda) : M \neq M' \wedge H^P(M) = H^P(M') \right],$$

$$\text{Adv}_{H, \mathcal{A}}^{\text{pre}}(\lambda) := \mathbb{P}_P \left[M \stackrel{\$}{\leftarrow} \mathbb{F}_2^\infty; h \leftarrow H^P(M); M' \leftarrow \mathcal{A}^{H^P, P^\pm}(1^\lambda, h) : H^P(M') = h \right],$$

$$\text{Adv}_{H, \mathcal{A}}^{\text{spre}}(\lambda) := \mathbb{P}_P \left[M \stackrel{\$}{\leftarrow} \mathbb{F}_2^\infty; M' \leftarrow \mathcal{A}^{H^P, P^\pm}(1^\lambda, M) : M \neq M' \wedge H^P(M) = H^P(M') \right].$$

In the above, the subscript P indicates that the randomness is taken over the uniform choice of P in $\text{Perm}(b)$, in addition to any internal randomness of $\mathcal{A}$. We say that H^P is **collision-resistant** (resp., **preimage-resistant**, **second-preimage-resistant**) if for any adversary $\mathcal{A}$ making at most q queries, the corresponding advantage $\text{Adv}_{H, \mathcal{A}}^{\text{coll}}(\lambda)$ (resp., $\text{Adv}_{H, \mathcal{A}}^{\text{pre}}(\lambda)$, $\text{Adv}_{H, \mathcal{A}}^{\text{spre}}(\lambda)$) is negligible in λ .

Remark 2.1 (Random Oracle Immunity). Let $\text{RO} \xleftarrow{\$} \text{Fun}(\mathbb{F}_2^\infty, \mathbb{F}_2^n)$ be a random oracle and let $\mathcal{A}$ be an adversary that makes at most q queries to RO. The three advantage notions then satisfy

$$\text{Adv}_{\text{RO}, \mathcal{A}}^{\text{coll}}(\lambda) \leq \frac{q(q-1)}{2^{n+1}}, \quad \text{Adv}_{\text{RO}, \mathcal{A}}^{\text{pre}}(\lambda) \leq \frac{q}{2^n}, \quad \text{Adv}_{\text{RO}, \mathcal{A}}^{\text{spre}}(\lambda) \leq \frac{q}{2^n}.$$

These bounds serve as the baseline against which the security of Laurus is measured in the ideal permutation model.

2.3 Indifferentiability Framework

For a permutation-based hash function H , let H^P denote the instance that has access to the idealized primitive $P \xleftarrow{\$} \text{Perm}(b)$, and let RO denote a random oracle with the same domain and range as H^P . The indifferentiability advantage of a distinguisher $\mathcal{D}$ against a simulator Sim is defined as

$$\text{Adv}_{H^P, \mathcal{D}}^{\text{RO}, \text{Sim}}(\lambda) := \left| \mathbb{P}[\mathcal{D}^{H^P, P^\pm}(1^\lambda) = 1] - \mathbb{P}[\mathcal{D}^{\text{RO}, \text{Sim}^\pm}(1^\lambda) = 1] \right|.$$

Here, Sim has oracle access to the random oracle RO and shares the same domain and range as P . The notation $\mathcal{D}^{\mathcal{L}, \mathcal{R}} = 1$ signifies that the distinguisher $\mathcal{D}$ outputs 1, indicating a decision in favor of the pair $(\mathcal{L}, \mathcal{R})$ being the real world (H^P, P) .

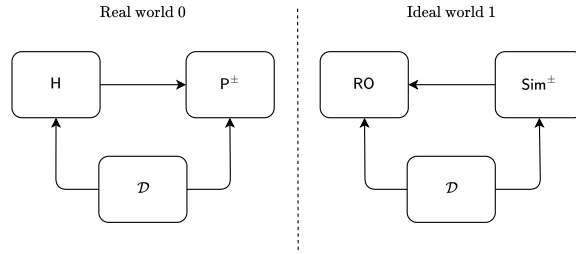


Figure 2. The indifferentiability framework. In the real world (left), the distinguisher $\mathcal{D}$ interacts with H^P and P . In the ideal world (right), $\mathcal{D}$ interacts with RO and a simulator Sim that has oracle access to RO.

We say a hash function H^P is $(t_{\mathcal{D}}, t_{\text{Sim}}, q, \varepsilon)$ -**indifferentiable** from a random oracle RO if there exists a simulator Sim with running time t_{Sim} such that for any distinguisher $\mathcal{D}$ with running time $t_{\mathcal{D}}$ making at most q total queries,

$$\text{Adv}_{H^P, \mathcal{D}}^{\text{RO}, \text{Sim}}(\lambda) \leq \varepsilon.$$

In particular, if there exists a simulator $\tilde{\text{S}}$ such that for all bounded distinguishers $\mathcal{D}$, H is $(t_{\mathcal{D}}, t_{\tilde{\text{S}}}, q^{\tilde{\text{S}}}, \varepsilon^{\tilde{\text{S}}})$ -indifferentiable from RO, where $q^{\tilde{\text{S}}} = \mathcal{O}(2^{b_H})$ and $\varepsilon^{\tilde{\text{S}}}$ is a negligible function in λ that does not depend on $\mathcal{D}$, then we call $b_H = b_H(\tilde{\text{S}})$ the **indifferentiability security bound** of H .

Remark 2.2. Since P is an idealized permutation, we formally denote P^+ (resp. P^-) as the forward (resp. backward) access to P , implemented by lazy sampling in the classical sense. Correspondingly, Sim is viewed as a pair $\text{Sim} = (\text{Sim}^+, \text{Sim}^-)$.

For the VFB construction [LLHL26], which serves as the foundation of Laurus's operation mode, the following indifferentiability bound has been established; see [LLHL26, Sec. 4] for the detailed definition of Venus's function $\mathcal{V}(c)$.

Theorem 2.1 (Indifferentiability of VFB [LLHL26, Theorem 1]). Let $P \xleftarrow{\$} \text{Perm}(b)$ be an ideal permutation of width $b = c + r$ with $r \geq c$, and let n be the output length of the VFB mode with

capacity c . There exists a simulator Sim such that for any distinguisher $\mathcal{D}$ making at most q queries with maximum message length $\ell_{\max}$,

$$\text{Adv}_{\text{VFB}, \mathcal{P}, \mathcal{D}}^{\text{RO}, \text{Sim}}(\lambda) \leq \frac{C \cdot (\ell_{\max} + L)q}{2^{\mathcal{V}(c)}},$$

where C is an absolute constant and $L = \lceil n/r \rceil$. Asymptotically, $\mathcal{V}(c) \rightarrow c$, so VFB achieves an asymptotically optimal indistinguishability security bound.

3 The Operation Mode Laurus $^\delta$

We now specify the building blocks of Laurus $^\delta[c, r, n]$. All security claims are made in the random oracle model, in which the underlying primitive is an idealized permutation $\mathcal{P}$ randomly sampled from $\text{Perm}(b)$. Working in this model avoids trivial collision attacks by non-uniform adversaries in the standard model, and allows us to derive collision, preimage, and second-preimage resistance directly from the indistinguishability bound.

The system parameters of Laurus are independent of the security parameter λ . The permutation width b is partitioned as $b = \theta + b'$, where θ is the length of the counter part. The remaining b' bits are further split as $b' = c' + r'$, where c' is the length of the internal state participating in the finalization, and r' is the length of the last message block.

3.1 Building Blocks

The components of Laurus $^\delta[c, r, n]$ are defined below, together with their procedural pseudocode in Fig. 3.

Setup	Ref	cnt^{t_M}
Input: λ 1. $c \leftarrow \lambda$ 2. $r \leftarrow b' - c$ 3. return (c, r)	Input: $x \in \mathbb{F}_2^{b'}$ 1. $x_1 \ x_2 \ x_3 \leftarrow x, \quad x_1, x_3 \in \mathbb{F}_2^{c'}$ 2. return $x_3 \ x_2 \ x_1$	Input: x, y 1. if $x = 0$ then return $0 \ (y)_2$ 2. else return $1 \ t_M \ (y)_2$
Opad	$\text{tr}_d, \text{Tr}_d, \text{Mix}$	$\text{TDM}_{x,y}^{\mathcal{P}}(c)$
Input: M 1. $t \leftarrow \min\{t \in \mathbb{N} : M + t \equiv r' \pmod{r}\}$ 2. $t_M \leftarrow \delta_0(t)$ 3. if $t = 0$ then $M^* \leftarrow M$ 4. else $M^* \leftarrow M \ 1 \ 0^{t-1}$ 5. $M_0 \ \dots \ M_{\ell-1} \ M_\ell \leftarrow M^*,$ $M_i \in \mathbb{F}_2^r \ (0 \leq i < \ell), M_\ell \in \mathbb{F}_2^{r'}$ 6. return $((M_0, \dots, M_\ell), t_M)$	Input: $\mathbf{x} = (x_0, \dots, x_{n-1})$ 1. $\text{tr}_d(\mathbf{x}) = (x_d, \dots, x_{n-1})$ 2. $\text{Tr}_d(\mathbf{x}) = (x_0, \dots, x_{d-1})$ Input: $S \in \mathbb{F}_2^{b'}, M \in \mathbb{F}_2^{r'}$ 3. $\text{Mix}(S, M) = \text{Tr}_{r'}(S) \ M \ \text{tr}_{c'}(S)$	Input: S 1. $S \leftarrow \text{cnt}^{t_M}(x, y) \ S$ 2. $S \leftarrow \mathcal{P}(S) + 0^{b-c} \ \text{tr}_{b-c}(S)$ 3. $S \leftarrow \text{tr}_\theta(S)$ 4. return S

Figure 3. Procedural summary of the building blocks.

Setup (Fig. 3, top left). Given λ , the function Setup sets $c \leftarrow \lambda$ and $r \leftarrow b' - c$. Here c simultaneously controls the feed-forward length and the fraction of the internal state that participates in compression, while r determines the size of every message block except the tail.

Reflection function (Fig. 3, top middle). $\text{Ref} : \mathbb{F}_2^{b'} \rightarrow \mathbb{F}_2^{b'}$ splits its input into $x = x_1 \| x_2 \| x_3$ where $|x_1| = |x_3| = r'$ and $|x_2| = b' - 2r'$, and returns $\text{Ref}(x) = x_3 \| x_2 \| x_1$.

Counter encoding (Fig. 3, top right). $\text{cnt}^{t_M} : \{0, 1\} \times \mathbb{N} \rightarrow \mathbb{F}_2^\infty$ encodes a mode bit x and a counter y as $\text{cnt}^{t_M}(0, y) = 0 \| (y)_2$ and $\text{cnt}^{t_M}(1, y) = 1 \| t_M \| (y)_2$, where $(y)_2$ is the binary representation of y .

Original padding (Fig. 3, bottom left). $\text{Opad} : \mathbb{F}_2^\infty \rightarrow (\mathbb{F}_2^r)^\infty \times \mathbb{F}_2^{r'} \times \mathbb{F}_2$ appends a single 1 followed by the minimal number of zeros so that $|M| + t \equiv r' \pmod{r}$. In particular, if no padding bits are needed, i.e., $t = 0$, the message is left unchanged. It then splits the result into r -bit blocks and a trailing r' -bit tail, and returns the block sequence together with $t_M = \delta_0(t)$ where $\delta_0(t) = 1$ if $t = 0$ and 0 otherwise.

Truncation and mixing (Fig. 3, bottom middle). For $d \leq n$, $\text{tr}_d : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^{n-d}$ drops the leftmost d bits, while $\text{Tr}_d : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^d$ retains the leftmost d bits. The mixing function $\text{Mix} : \mathbb{F}_2^{b'} \times \mathbb{F}_2^{r'} \rightarrow \mathbb{F}_2^{b'}$ interleaves appropriate truncations of the state S with a message block M ,

$$\text{Mix}(S, M) = \text{Tr}_{r'}(S) \| M \| \text{tr}_{c'}(S).$$

Truncated Davies–Meyer (Fig. 3, bottom right). For $x \in \{0, 1\}$ and a counter $y \in \mathbb{N}$, $\text{TDM}_{x,y}^P(c) : \mathbb{F}_2^{b'} \rightarrow \mathbb{F}_2^{b'}$ prepends a counter block to the state, applies P with a c -bit feed-forward, and truncates θ bits from the left:

$$[\text{TDM}_{x,y}^P(c)](S) = \text{tr}_\theta(P(\text{cnt}^{t_M}(x, y) \| S)) + 0^r \| \text{tr}_r(S).$$

3.2 Main Construction

With the above building blocks and notation in place, the complete specification of $\text{Laurus}^\delta[c, r, n]$ is given in Alg. 1.

Equivalent padding. Observe that in each compression round the only parts of the input to P that do not depend on the previous internal state are $\text{cnt}(0, i)$ and M_i . Hence we can view Opad together with the counter as an equivalent padding

$$\text{pad} : \mathbb{F}_2^\infty \rightarrow (\mathbb{F}_2^{\theta+r})^\infty \times \mathbb{F}_2^{\theta+r'}, \quad M \mapsto ((\text{cnt}^{t_M}(0, 0) \| M_0, \dots, \text{cnt}^{t_M}(0, \ell-1) \| M_{\ell-1}), \text{cnt}^{t_M}(1, \delta) \| M_\ell).$$

This map is injective since $\text{pad}(M) = \text{pad}(M')$ implies $t_M = t_{M'}$ and $M = M'$ by definition of Opad . Since the last block M_ℓ consists solely of the trailing message bits and the 10^* padding (if any), pad is a θ -saturated padding. Thus $\text{Laurus}^\delta[c, r, n]$ is a well-defined iterated hash function with padding pad , compression function $F : \mathbb{F}_2^{c+r} \times \mathbb{F}_2^{\theta+r} \rightarrow \mathbb{F}_2^{c+r}$ and chop function $G : \mathbb{F}_2^{c+r} \times \mathbb{F}_2^{\theta+r'} \rightarrow \mathbb{F}_2^n$. For a state $S \in \mathbb{F}_2^{c+r}$ and a padded message block M , the compression and chop functions are defined stepwise as follows.

Algorithm 1 The Laurus ^{δ} $[c, r, n]$ Hash Function

Input: $n \in \{512, 768, 1024\}$, $\lambda \in \{512, 768, 1024\}$, $(IV, IV') \in \mathbb{F}_2^{c+r}$, $M \in \mathbb{F}_2^\infty$

Output: $h \in \mathbb{F}_2^n$

```
1:  $(c, r) \leftarrow \text{Setup}(\lambda)$  /* parameter setup */
2:  $((M_0, \dots, M_{\ell-1}, M_\ell), t_M) \leftarrow \text{Opad}(M)$ 
3:  $S_0 \leftarrow (IV, IV')$ 
4: for  $i = 0$  to  $\ell - 1$  do /* compression */
5:    $(U_i^1, U_i^r) \leftarrow (\text{Tr}_r(\text{Ref}(S_i)), \text{tr}_r(\text{Ref}(S_i)))$ 
6:    $V_i \leftarrow \text{TDM}_{0,i}^P(c)(U_i^r \| M_i)$ 
7:    $S_{i+1} \leftarrow V_i + U_i^1 \| 0^c$ 
8:  $U \leftarrow \text{Mix}(S_\ell, M_\ell)$  /* finalization */
9:  $V \leftarrow \text{TDM}_{1,\delta}^P(c)(U)$  /*  $M_\ell$  does not participate in the feed-forward */
10:  $h \leftarrow \text{tr}_{b'-n}(V)$ 
11: return  $h$ 
```

Compression function $F(S, M)$.

$$\begin{aligned} (U^1, U^r) &\leftarrow (\text{Tr}_r(\text{Ref}(S)), \text{tr}_r(\text{Ref}(S))), \\ X &\leftarrow \text{Tr}_\theta(M) \| U^r \| \text{tr}_\theta(M), \\ V &\leftarrow P(X) + 0^{b-c} \| \text{tr}_{b-c}(X), \\ F(S, M) &\leftarrow \text{tr}_\theta(V) + U^1 \| 0^c. \end{aligned}$$

Chop function $G(S, M)$.

$$\begin{aligned} U &\leftarrow \text{Mix}(S, M), \\ X &\leftarrow \text{Tr}_\theta(M) \| U, \\ V &\leftarrow P(X) + 0^{b-c} \| \text{tr}_{b-c}(X), \\ G(S, M) &\leftarrow \text{tr}_{b'-n}(V). \end{aligned}$$

3.3 Generalized VFB Mode

Like the sponge construction, the VFB mode [LLHL26, Sec. 4.3] partitions the permutation state b into a capacity c and a rate r , and maintains a compact internal state of only $2c$ bits during compression. After each permutation call, VFB truncates the $(b - 2c)$ bits that are not needed for the next round. For a state $S \in \mathbb{F}_2^{c+r}$ and a message block $M \in \mathbb{F}_2^r$, the compression and chop functions of VFB are defined as follows.

Compression function $F_V(S, M)$.

$$\begin{aligned} (U^1, U^r) &\leftarrow (\text{tr}_c(S), \text{Tr}_r(S)), \\ X &\leftarrow U^r \| M, \\ Y &\leftarrow \text{Tr}_{2c}(P(X) + 0^c \| M), \\ F_V(S, M) &\leftarrow Y + U^1 \| 0^c. \end{aligned}$$

Chop function $G_V(S, M)$.

$$\begin{aligned} X &\leftarrow S \parallel M, \\ V &\leftarrow P(X) + X, \\ G_V(S, M) &\leftarrow \text{tr}_c(V). \end{aligned}$$

Recall that the indistinguishability proof of VFB relies on a suitable classification of **data pairs** in the partial graph technique [LLHL26, Sec. 4]. The security of a permutation-based hash mode is governed by a family of bad events whose probabilities depend on the structure of its data pairs, i.e., the portions of the primitive’s input/output pairs that participate in the iterative compression and are visible from the adversary’s perspective.

Let (X, Y) with $Y = P(X)$ denote an input/output pair of P . From the definitions of F_V and G_V , the information about (X, Y) that an adversary can extract during its interaction with the VFB mode is limited to a few specific data components. Concretely, for VFB the **data pair** associated with each permutation call is defined as

$$x^1 = X[1, c], \quad x^r = X[c + 1, c + r], \quad y^1 = Y[1, c], \quad y^r = (X + Y)[c + 1, 2c].$$

Proposition 3.1 (Data pair classification [PHG12, Sec. 11.2]). For a permutation-based hash function, the adversary can obtain partial or full information about a data pair $(X, Y) = (x^1 \parallel x^r, y^1 \parallel y^r)$ via four types of queries: forward P^+ , backward P^- , a digest block z_δ , or the internal state of an $\mathcal{L}$ -query. Based on the known or unknown status of each component, there are exactly six types of data pairs, denoted by $\{Q_j\}_{j=1}^6$.

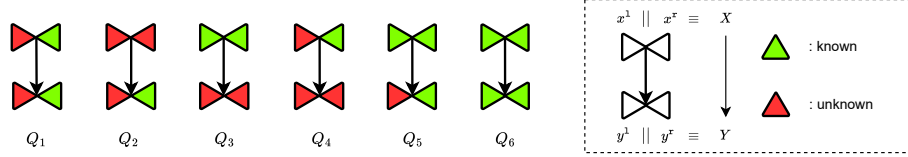


Figure 4. The six types of data pairs, based on the known or unknown status of x^1, x^r, y^1, y^r .

The data pair of Laurus^δ. To achieve high security under the NGCC parameter requirements, Laurus retains a larger internal state of $(c + r)$ bits without truncation beyond the counter block cnt . In our reference instantiation, P is a permutation of width $b = 1600$ bits, partitioned as $b = \theta + b'$ with counter length $\theta = 64$. Hence $b' = b - \theta = 1536$ bits, comprising $c' = 1024$ and $r' = 512$. By Setup, the rate satisfies $r = b' - c = b' - \lambda$, giving $r = 1024$ for $\lambda = 512$, $r = 768$ for $\lambda = 768$, and $r = 512$ for $\lambda = 1024$ (see Tab. 1).

Table 1. Parameter comparison between VFB and Laurus^δ at $\lambda \in \{512, 768, 1024\}$.

Scheme	λ	b	θ	b'	c'	r'	c	r
VFB	512	1600	—	—	—	—	512	1088
Laurus ^δ	512	1600	64	1536	1024	512	512	1024
Laurus ^δ	768	1600	64	1536	1024	512	768	768
Laurus ^δ	1024	1600	64	1536	1024	512	1024	512

From the definition of TDM, the primitive P is invoked in two distinct contexts. In the compression phase,

$$X = \text{cnt}(0, i) \parallel \text{tr}_r(\text{Ref}(S)) \parallel M, \quad V = 0^{b-c} \parallel \text{tr}_{b-c}(X + Y).$$

In the finalization phase,

$$X = \text{cnt}(1, \delta) \parallel \text{Tr}_{r'}(S) \parallel M_\ell \parallel \text{tr}_{c'}(S), \quad h = \text{tr}_{b-n}(X + Y).$$

Using the same notation, the data pair of Laurus^δ can be expressed as

$$\begin{aligned} x^1 &= X[\theta + 1, \theta + r'], & x^r &= X[\theta + c + 1, \theta + b'], \\ y^1 &= Y[\theta + 1, \theta + r'], & y^r &= (X + Y)[\theta + c' + 1, \theta + b']. \end{aligned}$$

Table 2. Concrete data-pair definitions for the reference instantiation.

Scheme	λ	x^1	x^r	y^1	y^r
VFB	512	$X[1, 512]$	$X[513, 1600]$	$Y[1, 512]$	$(X+Y)[513, 1024]$
Laurus^δ	512	$X[65, 576]$	$X[577, 1600]$	$Y[65, 576]$	$(X+Y)[1089, 1600]$
Laurus^δ	768	$X[65, 576]$	$X[833, 1600]$	$Y[65, 576]$	$(X+Y)[1089, 1600]$
Laurus^δ	1024	$X[65, 576]$	$X[1089, 1600]$	$Y[65, 576]$	$(X+Y)[1089, 1600]$

Remark 3.1. Observe that $\min\{|x^r|\} \geq 512$ and $|x^1| = |y^1| = |y^r| = 512$ for any security parameter $\lambda_{\text{Laurus}} \in \{512, 768, 1024\}$.

Generally speaking, a permutation-based hash whose data pairs take the form described in [Prop. 3.1](#) is called an HVFB mode, where “H” derives from a medical adjective meaning *hyperplastic*, signifying a more elaborate variant of the VFB mode.

[Tab. 2](#) and [Rem. 3.1](#) show that, under our parameter settings, Laurus^δ at any security level shares the same data-pair lengths as VFB at security level 512. A direct verification confirms that (x^1, x^r, y^1, y^r) follow exactly the same propagation paths during the iteration. Consequently, all bad events defined for the VFB indistinguishability proof at security level 512 carry over identically to Laurus^δ , and we obtain the following corollary.

Corollary 3.1. Under the same set of bad events that define the indistinguishability bound of VFB, the indistinguishability bound of Laurus is governed by probabilistic events occurring over the corresponding data-pair space. For $\lambda = n \in \{512, 768, 1024\}$ the indistinguishability bound of Laurus does not exceed that of VFB, i.e.,

$$\text{Adv}_{\text{Laurus}^\delta[c, r, n], P, \mathcal{D}}^{\text{RO, Sim}}(\lambda) \leq \text{Adv}_{\text{VFB}, P, \mathcal{D}}^{\text{RO, Sim}}(512) \leq \frac{C \cdot \ell_{\max} \cdot q}{2^{\mathcal{V}(512)}}.$$

Similarly, for $\lambda = 512$ and $n = 1024$, we have

$$\text{Adv}_{\text{Laurus}^\delta[512, 1024, 1024], P, \mathcal{D}}^{\text{RO, Sim}}(512) \leq \text{Adv}_{\text{VFB}, P, \mathcal{D}}^{\text{RO, Sim}}(512) \leq \frac{C \cdot \ell_{\max} \cdot q}{2^{\mathcal{V}(512)}}.$$

Corollary 3.2 (Security Properties of $\text{Laurus}^\delta[c, r, n]$). Combining the indistinguishability bound with the standard random-oracle bounds (see [Rem. 2.1](#)), for any adversary $\mathcal{A}$ making at most q queries with maximum message length $\ell_{\max}$, the collision, preimage, and second-preimage advantages of Laurus^δ satisfy

$$\begin{aligned} \text{Adv}_{\text{Laurus}^\delta[512, 1024, 512], \mathcal{A}}^{\text{coll}}(512) &\leq \frac{q^2}{2^{513}} + \frac{C \cdot \ell_{\max} \cdot q}{2^{\mathcal{V}(512)}}, \\ \text{Adv}_{\text{Laurus}^\delta[768, 768, 768], \mathcal{A}}^{\text{coll}}(768) &\leq \frac{q^2}{2^{769}} + \frac{C \cdot \ell_{\max} \cdot q}{2^{\mathcal{V}(512)}}, \end{aligned}$$

$$\begin{aligned}\text{Adv}_{\text{Laurus}^\delta[1024,512,1024],\mathcal{A}}^{\text{coll}}(1024) &\leq \frac{q^2}{2^{1025}} + \frac{C \cdot \ell_{\max} \cdot q}{2^{\mathcal{V}(512)}}, \\ \text{Adv}_{\text{Laurus}^\delta[512,1024,512],\mathcal{A}}^{\text{pre}}(512) &\leq \frac{q}{2^{512}} + \frac{C \cdot \ell_{\max} \cdot q}{2^{\mathcal{V}(512)}}, \\ \text{Adv}_{\text{Laurus}^\delta[512,1024,512],\mathcal{A}}^{\text{spre}}(512) &\leq \frac{q}{2^{512}} + \frac{C \cdot \ell_{\max} \cdot q}{2^{\mathcal{V}(512)}}.\end{aligned}$$

That is, $\text{Laurus}^\delta[512, 1024, 512]$ achieves 256-bit collision resistance, 512-bit (second) preimage resistance, and 512-bit indistinguishability security; $\text{Laurus}^\delta[768, 768, 768]$ achieves 384-bit collision resistance; and $\text{Laurus}^\delta[1024, 512, 1024]$ achieves 512-bit collision resistance.

4 Remaining Security Proofs

4.1 Collision Resistance for $\lambda = 1024$

For $\lambda = 1024$, the Setup procedure sets $n = c = \lambda = 1024$ and $r = 512$. The collision resistance bound obtained above via the VFB reduction is constrained by the function $\mathcal{V}(c)$ and its asymptotic nature. To eliminate any concern about concrete tightness, we additionally provide a direct proof of collision resistance in the ideal permutation model via a dedicated argument.

Primitive Transcripts and State Transcripts. For an iterated hash function H^P defined by a compression function F and a chop function G , denote by $\{(X_i(M), Y_i(M))\}_{i=1}^\ell$ the sequence of input–output pairs of the primitive P that are generated during the evaluation of $H(M)$. We call this sequence the *primitive transcript* of M with respect to H . Denote by $\{S_i(M)\}_{i=0}^\ell$ the sequence of internal states produced by iterating F , and call it the *internal state transcript* of M .

Classification of Collisions. If distinct messages $M \neq M'$ satisfy $H(M) = H(M')$, then by definition

$$G(S_\ell(M), M_\ell) = G(S_\ell(M'), M'_\ell).$$

We partition the collision event into two disjoint cases according to whether the final internal states differ:

- $\text{Coll}_{F,\mathcal{A}}$ (*collision in the final*): $\mathcal{A}$ outputs $M' \neq M$ such that $H(M) = H(M')$ and $S_\ell(M) \neq S_\ell(M')$;
- $\text{Coll}_{M,\mathcal{A}}$ (*collision in the middle*): $\mathcal{A}$ outputs $M' \neq M$ such that $H(M) = H(M')$ and $S_\ell(M) = S_\ell(M')$.

Every collision belongs to exactly one of these two categories, hence

$$\text{Adv}_{\text{Laurus}^\delta[c,r,n],\mathcal{A}}^{\text{coll}}(1024) = \mathbb{P}[\text{Coll}_{F,\mathcal{A}}] + \mathbb{P}[\text{Coll}_{M,\mathcal{A}}].$$

We bound each term separately in the following lemmas.

Lemma 4.1. $\mathbb{P}[\text{Coll}_{F,\mathcal{A}}] \leq q^2/2^{1025}$.

Proof. If $\mathcal{A}$ can trigger $\text{Coll}_{F,\mathcal{A}}$ within q queries, then we can construct an adversary $\mathcal{B}$ that finds a collision for $\text{tr}_{512}(\text{TDM}^P(1024))$ within q queries. For any input–output pair (X, Y) of the permutation P and any value h , the output of the truncated TDM satisfies

$$\mathbb{P}[\text{tr}_{512}(\text{TDM}^P(1024)(S)) = h] = \mathbb{P}[\text{tr}_{512}(X + Y) = h] = \frac{1}{2^{1024}}.$$

Indeed, whether (X, Y) is obtained via a forward or a backward query to P , the sum $(X + Y)$ is uniformly distributed over $\mathbb{F}_2^b$. Hence, by a standard birthday argument over the q queries,

$$\mathbb{P}[\text{Coll}_{F, \mathcal{A}}] \leq \sum_{i=1}^q \frac{i-1}{2^{1024}} \leq \frac{q^2}{2^{1025}}.$$

■

Lemma 4.2. $\mathbb{P}[\text{Coll}_{M, \mathcal{A}}] \leq \frac{q}{2^{1024}} + \frac{q^2}{2^{1025}}.$

Proof. Suppose $\mathcal{A}$ outputs $M \neq M'$ triggering $\text{Coll}_{M, \mathcal{A}}$, i.e.,

$$H(M) = H(M') \text{ and } S_\ell(M) = S_{\ell'}(M').$$

Then there exist indices $t \in [0, \ell]$ and $t' \in [0, \ell']$ such that $S_t(M) = S_{t'}(M')$ with $(t, t') \neq (0, 0)$ in the respective internal state transcripts. Choose the pair (t, t') with minimal t .

Case $t = 0$. Then $S_t = (IV, IV')$ and $t' \geq 1$. Define the *preimage-finding event*

$$\text{Pre}_{(IV, IV'), \mathcal{A}}^F : \mathcal{A} \text{ outputs } (S, M) \text{ such that } F(S, M) = (IV, IV').$$

Here $S'_{t'-1}$ witnesses exactly $\text{Pre}_{(IV, IV'), \mathcal{A}}^F$. By the uniform distribution of $\text{tr}_{512}(X + Y)$, each query succeeds with probability at most $1/2^{1024}$, giving

$$\mathbb{P}[\text{Pre}_{(IV, IV'), \mathcal{A}}^F] \leq \sum_{i=1}^q \frac{1}{2^{1024}} = \frac{q}{2^{1024}}.$$

Case $t \geq 1$. By the minimality of t , we must have

$$(X_t(M), Y_t(M)) \neq (X_{t'}(M'), Y_{t'}(M')).$$

Otherwise, by definition of F , we would obtain $S_{t-1}(M) = S_{t'-1}(M')$, contradiction. Define the *intermediate-collision event*

$$\text{Coll}_{M, \mathcal{A}}^d : \text{Coll}_{M, \mathcal{A}} \text{ occurs and } (X_t(M), Y_t(M)) \neq (X_{t'}(M'), Y_{t'}(M')).$$

By a birthday argument over the q queries,

$$\mathbb{P}[\text{Coll}_{M, \mathcal{A}}^d] \leq \mathbb{P}[\text{tr}_{512}(X_t(M) + Y_t(M)) = \text{tr}_{512}(X_{t'}(M') + Y_{t'}(M'))] \leq \sum_{i=1}^q \frac{i-1}{2^{1024}} \leq \frac{q^2}{2^{1025}}.$$

Summing the two disjoint cases yields the claimed bound. ■

Combining Lemma 4.1 and Lemma 4.2, we obtain the following theorem.

Theorem 4.1 (Collision Resistance of $\text{Laurus}^\delta[c, r, n]$ with $n = \lambda$). Let H^P be the $\text{Laurus}^\delta[c, r, n]$ hash function with $n = \lambda = 1024$. For any adversary $\mathcal{A}$ making at most q queries,

$$\text{Adv}_{H^P, \mathcal{A}}^{\text{coll}}(1024) \leq \frac{q}{2^{1024}} + \frac{q^2}{2^{1024}}.$$

Hence, for $n = \lambda = 1024$, $\text{Laurus}^\delta[c, r, n]$ provides 512 bits of collision resistance.

4.2 (Second) Preimage Resistance for $\lambda = 512, 768, 1024$

Theorem 4.2 (Preimage Resistance of $\text{Laurus}^\delta[c, r, n]$ with $n = \lambda$). Let H^P be the $\text{Laurus}^\delta[c, r, n]$ hash function with $n = \lambda \in \{512, 768, 1024\}$. For any adversary $\mathcal{A}$ making at most q queries,

$$\text{Adv}_{H^P, \mathcal{A}}^{\text{pre}}(\lambda) \leq \frac{q}{2^\lambda}.$$

Proof. Let $h \in \mathbb{F}_2^n$ be the target digest. If $\mathcal{A}$ outputs M such that $H(M) = h$, then the internal state transcript of M yields a preimage (S_ℓ, M_ℓ) of h under the chop function G , i.e., $G(S_\ell, M_\ell) = h$. Define the event

$$\text{Pre}_{h, \mathcal{A}}^G : \mathcal{A} \text{ outputs } (S, M) \text{ such that } G(S, M) = h.$$

Hence

$$\text{Adv}_{H^P, \mathcal{A}}^{\text{pre}}(n) \leq \mathbb{P}[\text{Pre}_{h, \mathcal{A}}^G] \leq \sum_{i=1}^q \mathbb{P}[\text{tr}_n(X_i(M) + Y_i(M)) = h] \leq \frac{q}{2^n}.$$

Taking $n = \lambda$ yields the claimed result. $\blacksquare$

For second preimage resistance, the adversary is additionally given a target message M of length ℓ blocks. This message fixes ℓ input–output pairs (X_i, Y_i) in the primitive transcript. In particular, for every $i \in [\ell]$, the first θ bits of X_i satisfy $\text{Tr}_\theta(X_i) = \text{cnt}(0, i)$.

Suppose $\mathcal{A}$ outputs $M' \neq M$ such that $H(M') = H(M) = h$. We distinguish two cases according to whether the final internal states coincide.

Case 1: $S_\ell \neq S_\ell(M')$. Then $(S_\ell(M'), M'_\ell)$ constitutes a preimage of h under G , triggering $\text{Pre}_{h, \mathcal{A}}^G$.

Case 2: $S_\ell = S_\ell(M')$. There exist indices $t \in [0, \ell]$ and $t' \in [0, \ell']$ such that $S_t = S_{t'}(M')$ with $(t, t') \neq (0, 0)$. Choose the pair (t, t') with minimal t' .

Subcase 2.1: $t' = 0$. Then $S_t = (\text{IV}, \text{IV}')$, yielding a preimage of (IV, IV') under the compression function F .

Subcase 2.2: $t' > 0$. Then $S_{t'-1}(M')$ together with $M'_{t'}$ triggers $\text{Pre}_{S_t, \mathcal{A}}^F$. Summing the three mutually exclusive cases, we obtain the following theorem.

Theorem 4.3 (Second Preimage Resistance of $\text{Laurus}^\delta[c, r, n]$ with $n = \lambda$). Let H^P be the $\text{Laurus}^\delta[c, r, n]$ hash function with $n = \lambda \in \{512, 768, 1024\}$. For any adversary $\mathcal{A}$ making at most q queries against a target message of length ℓ blocks,

$$\text{Adv}_{H^P, \mathcal{A}}^{\text{spre}}(\lambda) \leq \mathbb{P}[\text{Pre}_{h, \mathcal{A}}^G] + \mathbb{P}[\text{Pre}_{(\text{IV}, \text{IV}'), \mathcal{A}}^F] + \mathbb{P}[\text{Pre}_{S_t, \mathcal{A}}^F] \leq \frac{2q}{2^\lambda} + \frac{\ell_{\max}}{2^\lambda}.$$

where $\ell_{\max}$ is the maximum message length in blocks.

Hence, $\text{Laurus}^\delta[c, r, n]$ achieves n -bit second preimage resistance for $n = \lambda \in \{512, 768, 1024\}$.

5 Conclusions

We have presented a comprehensive security analysis of the Laurus hash function. The Laurus hash is instantiated as $\text{Laurus}^0[c, r, \lambda]$ for $\lambda \in \{512, 768, 1024\}$, while Laurus-XOF is obtained by concatenating the outputs of $\text{Laurus}^\delta[512, 1024, 1024]$. By [Cor. 3.2](#), [Thm. 4.1](#), [Thm. 4.2](#), and [Thm. 4.3](#), Laurus achieves optimal collision, preimage, and second-preimage resistance at all three security levels. In particular, Laurus-512 and Laurus-XOF attain optimal 512-bit indistinguishability. [Tab. 3](#) compares Laurus with existing permutation-based hash functions.

Table 3. Security comparison of permutation-based hash functions. Bounds shown in gray are not proved directly but can be implied by the indifferntiability bound via the random-oracle reduction.

Construction	# Perm. / State	Rate	Collision	Preimage	2nd-Pre.	Indiff.
Sponge [BDPA08]	$1 / c + r$	r	$\min\{\frac{n}{2}, \frac{c}{2}\}$	n_{LM}	$\min\{n, \frac{c}{2}\}$	$\frac{c}{2}$
Sponge-F [GHJ ⁺ 25]	$1 / c + r$	r	$\frac{h}{2}$	h	$h(\ell)$	$\frac{c}{2}$
SPONGE-DM [SLZ ⁺ 25]	$1 / c + r$	r	$\min\{\frac{n}{2}, \frac{c}{2}\}$	n	$n(\ell)$	$\frac{c}{2}$
Double Sp. [LM24]	$2 / c + r$	r	-	-	-	$\frac{2c}{3}$
Alt. Sp. [LSY ⁺ 26]	$2 / c + 2r$	r	-	-	-	$\min\{\frac{2c}{3}, \frac{r+c}{2}\}$
JH [Wu11]	$1 / 2n$	n	$\frac{n}{2}$	-	-	$\frac{n}{2}$
Laurus	$1 / \theta + c + r$	r	$\frac{c}{2}$	c	c	$\mathcal{V}(c)$

Notes. For the Sponge, the preimage bound is $n_{\text{LM}} := \min\{n, \max\{\frac{c}{2}, n-r'\}\}$ [LM22]. For SPONGE-DM and Sponge-F, the second-preimage bound takes the common form $x(\ell) = \min\{x, c - \log_2(\ell_{\max})\}$, where x is the output length and $\ell_{\max}$ is the maximum message length in blocks.

Remarks on Post-Quantum Security. Although the proofs in this work are given in the classical ideal permutation model, the security of Laurus against quantum adversaries can be assessed via the quantum random oracle model (QROM). By Grover’s algorithm [Gro96], the Brassard–Høyer–Tapp (BHT) algorithm [BHT98], and Zhandry’s optimality results [Zha12], an n -bit hash function provides $n/3$ -bit quantum collision resistance and $n/2$ -bit quantum (second) preimage resistance. Conservatively, we therefore consider Laurus-512 to provide 128-bit collision and 256-bit preimage security; Laurus-768 to provide 192-bit collision and 384-bit preimage security; and Laurus-1024 to provide 256-bit collision and 512-bit preimage security against quantum adversaries. These levels meet or exceed the post-quantum security requirements of the NGCC program.

The key advantage of Laurus over competing designs is its ability to achieve beyond-birthday-bound indifferntiability at $\lambda = 512$ and $\lambda = 768$. As Tab. 3 shows, among all designs achieving beyond-birthday-bound indifferntiability, Laurus achieves the best trade-off in both internal state size and number of primitive calls. In conclusion, Laurus offers a compelling trade-off between security, efficiency, and design simplicity for the next-generation hash function standard.

References

- [BDPA07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. In *ECRYPT Hash Workshop*, Barcelona, Spain, May 2007.
- [BDPA08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferntiability of the sponge construction. In Nigel P. Smart, editor, *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 181–197. Springer, 2008.
- [BDPA09] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Keccak sponge function family – main document. Submission to NIST (Round 2), 2009.
- [BHT98] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In *Proceedings of the 3rd Latin American Symposium on Theoretical Informatics – LATIN 1998*, volume 1380 of *Lecture Notes in Computer Science*, pages 163–169. Springer, 1998.

- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  ffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34(3):33, 2021.
- [GHJ⁺25] Chun Guo, Kai Hu, Shuntian Jiang, Yanhong Fan, Yong Fu, Bart Preneel, and Meiqin Wang. Adding feeding forward back to the sponge construction. Cryptology ePrint Archive, Report 2025/1006, 2025.
- [Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing – STOC 1996*, pages 212–219. ACM, 1996.
- [Ins25] Institute of Commercial Cryptography Standards (ICCS). Submission requirements for cryptographic hash algorithms. Next-generation Commercial Cryptographic Algorithms Program (NGCC), October 2025.
- [LLHL26] Hongli Li, Changlun Li, Honggang Hu, and Fengmei Liu. Indifferentiability of the sponge hash family: New attacks and the optimal construction. *IACR Transactions on Symmetric Cryptology*, 2026(1):31–75, 2026.
- [LM22] Charlotte Lefevre and Bart Mennink. Tight preimage bounds for the (generalized) sponge construction. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022*, Lecture Notes in Computer Science. Springer, 2022.
- [LM24] Charlotte Lefevre and Bart Mennink. Permutation-based hashing beyond the birthday bound. *IACR Transactions on Symmetric Cryptology*, 2024(1):71–113, 2024.
- [LSY⁺26] Ziyang Luo, Yaobin Shen, Hailun Yan, Lei Wang, and Dawu Gu. Alternating sponge: A low-memory hash function with beyond-birthday-bound security. Cryptology ePrint Archive, Report 2026/707, 2026.
- [PHG12] Souradyuti Paul, Ekawat Homsirikamol, and Kris Gaj. A novel permutation-based hash mode of operation FP and the hash function SAMOSA. In Steven D. Galbraith and Mridul Nandi, editors, *Progress in Cryptology – INDOCRYPT 2012*, volume 7668 of *Lecture Notes in Computer Science*, pages 509–527. Springer, 2012.
- [SLZ⁺25] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance – Application to hash-based signature schemes. Cryptology ePrint Archive, Report 2025/963, 2025.
- [Wu11] Hongjun Wu. The hash function JH. Submission to NIST (Round 3), 2011.
- [Zha12] Mark Zhandry. How to construct quantum random functions. In *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 679–697. Springer, 2012.